

L E C T U R E N O T E S

CSD622 · LECTURE 2

Pattern Recognition and Data Mining

Deep Dive — Continuing from Lecture 1

Master of Science Programme

Tutor: Dr A.J Fofanah

Department of Computer Science

Topics Covered: Naive Bayes (Worked Example) · Decision Tree Information Gain · SVM
Lagrangian Derivation Neural Network Training · LSTMs · Transformers · GMMs · Spectral
Clustering · Statistical Evaluation

Table of Contents

Table of Contents	
Introduction to Lecture 2	5
How to Use These Notes	5
Section 2.1: Naive Bayes Classifier — In Depth	5
2.1.1 Derivation from Bayes' Theorem	5
2.1.2 Multinomial Naive Bayes for Text	5
Training Phase	
Prediction Phase	
2.1.3 Laplace Smoothing	5
2.1.4 Worked Example — Spam Classification	5
Training Corpus	5
Step 1 — Compute Priors	6
Step 2 — Build Vocabulary and Count Words	6
Step 3 — Compute Likelihoods with Laplace Smoothing ($\alpha = 1$)	6
Step 4 — Classify New Email: "free prize now"	
Step 5 — Decision	7
2.1.5 Gaussian Naive Bayes	7
2.1.6 Naive Bayes Variants Summary	8
Section 2.2: Decision Trees — Advanced Topics	9
2.2.1 Information Gain — Full Worked Example	9
Step 1 — Entropy of the Root Node	9
Step 2 — Information Gain for Outlook	9
Step 3 — Information Gain for Wind	9
Results Summary and Tree Construction	10
2.2.2 Gain Ratio — C4.5's Improvement	10
2.2.3 CART, ID3, and C4.5 — Detailed Comparison	10
2.2.4 Pruning Strategies	11
Pre-pruning (Early Stopping)	11
Post-pruning (Cost-Complexity Pruning — CART)	12
Section 2.3: Support Vector Machines — Mathematical Derivation	13
2.3.1 The Primal Problem	13
2.3.2 The Lagrangian and the Dual Problem	13
2.3.3 KKT Conditions and Support Vectors	14
2.3.4 Soft Margin SVM	14

2.3.5 Hyperparameter Tuning: C and γ	15
2.3.6 Multi-Class SVMs.....	15
2.3.7 Python Implementation Best Practices.....	15
Section 2.4: Training Neural Networks — Deep Dive.....	17
2.4.1 The Vanishing and Exploding Gradient Problems.....	17
Vanishing Gradients.....	17
Exploding Gradients.....	17
2.4.2 Optimisation Algorithms.....	18
Stochastic Gradient Descent (SGD).....	18
Momentum.....	18
Adam (Adaptive Moment Estimation).....	18
2.4.3 Weight Initialisation.....	19
2.4.4 Batch Normalisation.....	19
2.4.5 Dropout.....	20
Section 2.5: RNNs, LSTMs, and Transformer Models.....	21
2.5.1 Vanilla RNNs and Their Limitations.....	21
2.5.2 LSTM — Full Gate Equations.....	21
Forget Gate.....	21
Input Gate.....	21
Cell State Update.....	22
Output Gate.....	22
Parameter Count for LSTM.....	22
2.5.3 Gated Recurrent Unit (GRU).....	22
2.5.4 The Attention Mechanism.....	23
Scaled Dot-Product Attention.....	23
Multi-Head Attention.....	23
2.5.5 BERT — Bidirectional Encoder.....	24
Section 2.6: Advanced Clustering and Evaluation Methods.....	25
2.6.1 Gaussian Mixture Models (GMMs) and the EM Algorithm.....	25
The EM Algorithm for GMMs.....	25
2.6.2 Spectral Clustering.....	26
Algorithm — Four Steps.....	26
2.6.3 ROC vs Precision-Recall — When to Use Which.....	27
ROC Curve Analysis.....	27
Precision-Recall Curve.....	27
2.6.4 Statistical Significance Testing.....	28
McNemar's Test.....	28

k-Fold Cross-Validation t-Test.....	28
Friedman Test for Multiple Algorithms.....	28
Section 2.7: Feature Importance and Model Interpretation.....	30
2.7.1 Permutation Importance.....	30
2.7.2 SHAP Values.....	30
2.7.3 LIME.....	31
Section 2.8: Handling Imbalanced Datasets — Complete Guide.....	32
2.8.1 Diagnosing the Problem.....	32
2.8.2 Resampling Techniques.....	32
Random Under-sampling.....	32
SMOTE — Synthetic Minority Over-sampling Technique.....	32
2.8.3 Algorithmic Approaches.....	33
Class Weights.....	33
Threshold Tuning.....	33
Section 2.9: Advanced Association Rule Measures.....	35
2.9.1 Limitations of Basic Measures.....	35
Problems with Confidence.....	35
Problems with Lift.....	35
2.9.2 Additional Interestingness Measures.....	35
2.9.3 Rule Filtering Strategy.....	36
Section 2.10: End-to-End ML Pipeline — Best Practices.....	37
2.10.1 The Data Leakage Problem.....	37
2.10.2 Complete Pipeline Implementation.....	37
Revision Guide — Lecture 2.....	40
10.1 Key Concepts Checklist.....	40
Section 2.1 — Naive Bayes.....	40
Section 2.2 — Decision Trees.....	40
Section 2.3 — SVM.....	40
Section 2.4 — Neural Network Training.....	41
Section 2.5 — RNNs and Transformers.....	41
Section 2.6 — Advanced Clustering and Evaluation.....	41
10.2 Sample Examination Questions with Model Answers.....	42
Question 1 (12 marks).....	42
Question 2 (10 marks).....	42
Question 3 (12 marks).....	43
Question 4 (10 marks).....	43
Question 5 (8 marks).....	44

Question 6 (13 marks).....	44
10.3 Glossary of New Terms from Lecture 2.....	45

Introduction to Lecture 2

These notes form the companion document to CSD622 Lecture 2: Pattern Recognition and Data Mining — Deep Dive. They expand on every topic introduced in Lecture 1, providing full mathematical derivations, step-by-step worked examples, Python code implementations, and examination guidance.

Lecture 1 established the conceptual architecture of the course: the 7-stage pattern recognition pipeline, Bayesian decision theory, k-NN, decision trees at a high level, SVMs, ensemble methods, neural network fundamentals, unsupervised learning, association rules, feature engineering, evaluation metrics, and ethics. Lecture 2 goes substantially deeper into each of these areas, adding the rigour and worked detail that examination questions will demand.

How to Use These Notes

- Work through each section with pencil in hand — re-derive every formula to build genuine understanding
- Attempt the worked examples before reading the solutions
- After each section, attempt the corresponding lecture exam questions from the PPT slides
- Return to Lecture 1 notes whenever a foundational concept needs refreshing

 **Note:** These notes assume familiarity with Lecture 1 content. Definitions established there (feature vector, Bayes' theorem, k-NN algorithm, etc.) are not re-introduced but are built upon.

Section 2.1: Naive Bayes Classifier — In Depth

In Lecture 1 we stated the Naive Bayes decision rule and listed its three variants. Here we derive the full algorithm from first principles, work through a complete spam classification example with numerical calculations, and discuss the critical implementation detail of Laplace smoothing.

2.1.1 Derivation from Bayes' Theorem

Recall Bayes' theorem from Lecture 1:

$$P(c | x) = P(x | c) \cdot P(c) / P(x)$$

The 'naive' assumption is that all features $x_1, x_2, \dots, x_n$ are conditionally independent given the class c . This means the joint likelihood factorises:

$$P(x | c) = P(x_1, x_2, \dots, x_n | c) = \prod_j P(x_j | c)$$

Substituting into Bayes' theorem and noting that $P(x)$ is constant across classes (cancels in the argmax):

$$\hat{c} = \operatorname{argmax}_c P(c) \cdot \prod_j P(x_j | c)$$

In practice, computing the product of many small probabilities causes numerical underflow — the result rounds to zero. We therefore take logarithms, converting the product to a sum:

$$\hat{c} = \operatorname{argmax}_c [\log P(c) + \sum_j \log P(x_j | c)]$$

 **Note:** The log transformation is monotone — it preserves the argmax. The class with the highest log-score is identical to the class with the highest probability score.

2.1.2 Multinomial Naive Bayes for Text

For text classification, each document is represented as a bag of words — a vector of word counts. The Multinomial Naive Bayes model assumes each word is generated independently from a multinomial distribution parameterised per class.

Training Phase

1. Compute prior probability $P(c) = (\text{number of documents in class } c) / (\text{total documents})$

2. For each word w in the vocabulary, compute the class-conditional likelihood $P(w | c)$
 $= (\text{count of } w \text{ in class } c \text{ documents}) / (\text{total words in class } c \text{ documents})$
3. Apply Laplace smoothing to prevent zero probabilities (see 2.1.3)

Prediction Phase

4. Tokenise the new document into words
5. For each class c , compute: $\log P(c) + \sum_{\{w \text{ in doc}\}} \text{count}(w, \text{doc}) \times \log P(w | c)$
6. Assign the document to the class with the highest score

2.1.3 Laplace Smoothing

Without smoothing, if a word appears in a test document but never appeared in class c during training, then $P(w | c) = 0$, which zeroes out the entire posterior regardless of all other evidence. This is catastrophic.

Laplace smoothing (additive smoothing with parameter α , typically $\alpha = 1$) adds α pseudo-counts to every word in every class:

$$P(w_j | c) = (\text{count}(w_j, c) + \alpha) / (\sum_k \text{count}(w_k, c) + \alpha \cdot |V|)$$

Where $|V|$ is the vocabulary size (number of distinct words across all documents). This guarantees every probability is strictly positive, and the probabilities still sum to 1 over the vocabulary.

2.1.4 Worked Example — Spam Classification

This example works through the complete Naive Bayes classification process from training corpus to classifying a new email. Follow each numerical step carefully — this is exactly the type of calculation examined.

Training Corpus

ID	Document Text	Label
T1	free money now	Spam
T2	win a free prize	Spam
T3	meeting at noon	Ham

T4	project deadline tomorrow	Ham
T5	free lunch at noon	Ham

Step 1 — Compute Priors

Total documents = 5. Spam documents = 2, Ham documents = 3.

$$P(\text{Spam}) = 2/5 = 0.40$$

$$P(\text{Ham}) = 3/5 = 0.60$$

Step 2 — Build Vocabulary and Count Words

Vocabulary $V = \{\text{free, money, now, win, a, prize, meeting, at, noon, project, deadline, tomorrow, lunch}\} \rightarrow |V| = 13$

Words in Spam documents (T1, T2): free(2), money(1), now(1), win(1), a(1), prize(1) $\rightarrow$ Total spam word tokens = 7

Words in Ham documents (T3, T4, T5): meeting(1), at(2), noon(2), project(1), deadline(1), tomorrow(1), free(1), lunch(1) $\rightarrow$ Total ham word tokens = 10

Step 3 — Compute Likelihoods with Laplace Smoothing ($\alpha = 1$)

For each word, we compute $P(\text{word} | \text{class}) = (\text{count} + 1) / (\text{total_tokens_in_class} + |V|)$:

$$\text{Spam denominator} = 7 + 13 = 20$$

$$\text{Ham denominator} = 10 + 13 = 23$$

$$P(\text{"free"} | \text{Spam}) = (2+1)/20 = 3/20 = 0.150$$

$$P(\text{"prize"} | \text{Spam}) = (1+1)/20 = 2/20 = 0.100$$

$$P(\text{"now"} | \text{Spam}) = (1+1)/20 = 2/20 = 0.100$$

$$P(\text{"free"} | \text{Ham}) = (1+1)/23 = 2/23 = 0.087$$

$$P(\text{"prize"} | \text{Ham}) = (0+1)/23 = 1/23 = 0.043$$

$$P(\text{"now"} | \text{Ham}) = (0+1)/23 = 1/23 = 0.043$$

Step 4 — Classify New Email: "free prize now"

For each class, compute the log-score:

$$\log \text{ score}(\text{Spam}) = \log P(\text{Spam}) + \log P(\text{"free"}|\text{Spam}) + \log P(\text{"prize"}|\text{Spam}) + \log P(\text{"now"}|\text{Spam})$$

$$\begin{aligned} \log \text{ score}(\text{Spam}) &= \log(0.40) + \log(0.150) + \log(0.100) + \log(0.100) \\ &= -0.916 + (-1.897) + (-2.303) + (-2.303) \\ &= -7.419 \end{aligned}$$

$$\log \text{ score}(\text{Ham}) = \log P(\text{Ham}) + \log P(\text{"free"}|\text{Ham}) + \log P(\text{"prize"}|\text{Ham}) + \log P(\text{"now"}|\text{Ham})$$

$$\begin{aligned} \log \text{ score}(\text{Ham}) &= \log(0.60) + \log(0.087) + \log(0.043) + \log(0.043) \\ &= -0.511 + (-2.441) + (-3.147) + (-3.147) \\ &= -9.246 \end{aligned}$$

Step 5 — Decision

Since $-7.419 > -9.246$, the Spam class has the higher log-score. Therefore the email 'free prize now' is classified as SPAM.

Example — Interpretation: Even though the prior favours Ham (0.60 vs 0.40), the word likelihoods for 'prize' and 'now' are much higher for Spam than Ham — particularly 'prize' which never appeared in Ham training documents (without smoothing, $P(\text{prize}|\text{Ham}) = 0$ would make classification impossible).

2.1.5 Gaussian Naive Bayes

When features are continuous rather than word counts, we replace the multinomial likelihood with a Gaussian (Normal) distribution:

$$P(x_j | c) = (1/\sqrt{2\pi\sigma^2_{jc}}) \cdot \exp(-(x_j - \mu_{jc})^2 / (2\sigma^2_{jc}))$$

Training simply estimates the mean μ_{jc} and variance σ^2_{jc} of each feature j for each class c from the training data. Prediction uses the log of this Gaussian density in the same summation formula.

Example — Iris Dataset: The Iris dataset has 4 continuous features: sepal length, sepal width, petal length, petal width. For class 'Setosa', we estimate: $\mu = [5.006, 3.428, 1.462, 0.246]$ and σ^2 per feature. Given a new flower $x = [5.1, 3.5, 1.4, 0.2]$, we compute the Gaussian log-likelihood for each class and classify to the highest scorer.

2.1.6 Naive Bayes Variants Summary

Variant	Likelihood Model	Feature Type	Best Used For
Gaussian NB	Normal distribution $N(\mu, \sigma^2)$	Continuous real values	Numeric features, Iris, medical data
Multinomial NB	Multinomial distribution	Integer counts (e.g., word counts)	Text classification (standard choice)
Bernoulli NB	Bernoulli distribution	Binary (word present/absent)	Short documents, Boolean features
Complement NB	Per-class complement statistics	Integer counts	Imbalanced text datasets

 **Note:** Despite the strong conditional independence assumption — which is almost always violated in practice — Naive Bayes frequently achieves competitive classification performance. This is because even when probabilities are not well-calibrated, the ranking of classes (which is all argmax cares about) is often correct.

Section 2.2: Decision Trees — Advanced Topics

2.2.1 Information Gain — Full Worked Example

We use the classic Play Tennis dataset (Mitchell, 1997) with 14 samples and the binary target 'Play Tennis' (9 Yes, 5 No). We compute Information Gain for each feature to determine which attribute should be the root of the tree.

Step 1 — Entropy of the Root Node

With 9 positive (Yes) and 5 negative (No) samples out of 14 total:

$$H(S) = -(9/14) \cdot \log_2(9/14) - (5/14) \cdot \log_2(5/14)$$

$$\begin{aligned} &= -(0.643) \cdot \log_2(0.643) - (0.357) \cdot \log_2(0.357) \\ &= -(0.643)(-0.637) - (0.357)(-1.485) \\ &= 0.410 + 0.530 = 0.940 \text{ bits} \end{aligned}$$

Step 2 — Information Gain for Outlook

Outlook has three values: Sunny (5 samples: 2 Yes, 3 No), Overcast (4 samples: 4 Yes, 0 No), Rainy (5 samples: 3 Yes, 2 No).

$$\begin{aligned} H(\text{Sunny}) &= -(2/5) \log_2(2/5) - (3/5) \log_2(3/5) = 0.971 \text{ bits} \\ H(\text{Overcast}) &= -(4/4) \log_2(4/4) - (0/4) \log_2(0/4) = 0.000 \text{ bits (pure node!)} \\ H(\text{Rainy}) &= -(3/5) \log_2(3/5) - (2/5) \log_2(2/5) = 0.971 \text{ bits} \end{aligned}$$

$$IG(S, \text{Outlook}) = H(S) - [(5/14) \cdot H(\text{Sunny}) + (4/14) \cdot H(\text{Overcast}) + (5/14) \cdot H(\text{Rainy})]$$

$$\begin{aligned} &= 0.940 - [(5/14) \cdot 0.971 + (4/14) \cdot 0.000 + (5/14) \cdot 0.971] \\ &= 0.940 - [0.347 + 0.000 + 0.347] \\ &= 0.940 - 0.693 = 0.247 \text{ bits} \end{aligned}$$

Step 3 — Information Gain for Wind

Wind has two values: Weak (8 samples: 6 Yes, 2 No), Strong (6 samples: 3 Yes, 3 No).

$$\begin{aligned} H(\text{Weak}) &= -(6/8) \log_2(6/8) - (2/8) \log_2(2/8) = 0.811 \text{ bits} \\ H(\text{Strong}) &= -(3/6) \log_2(3/6) - (3/6) \log_2(3/6) = 1.000 \text{ bits} \end{aligned}$$

$$IG(S, \text{Wind}) = 0.940 - [(8/14) \cdot 0.811 + (6/14) \cdot 1.000] = 0.940 - 0.892 = 0.048 \text{ bits}$$

Results Summary and Tree Construction

Computing IG for all four features yields:

Feature	Information Gain	Decision
Outlook	0.247 bits	BEST SPLIT — becomes root node
Temperature	0.029 bits	Poor discriminator
Humidity	0.151 bits	Second choice
Wind	0.048 bits	Weak discriminator

Outlook is selected as the root. The Overcast branch is immediately a pure leaf (all 4 samples are Yes). The Sunny and Rainy branches are further split by recursively applying the same criterion to the remaining samples.

2.2.2 Gain Ratio — C4.5's Improvement

A fundamental bias of Information Gain is that it favours features with many distinct values. Imagine a feature 'StudentID' with one unique value per student — it achieves perfect split (IG = $H(S) = 0.940$ bits) but is completely useless for prediction on new data. The Gain Ratio correction penalises high-cardinality features:

$$\text{SplitInfo}(S, A) = -\sum_v (|S_v|/|S|) \cdot \log_2(|S_v|/|S|)$$

$$\text{GainRatio}(S, A) = \text{IG}(S, A) / \text{SplitInfo}(S, A)$$

For Outlook: $\text{SplitInfo} = -(5/14)\log_2(5/14) - (4/14)\log_2(4/14) - (5/14)\log_2(5/14) = 1.577$ bits

$$\text{GainRatio}(\text{Outlook}) = 0.247 / 1.577 = 0.157$$

A StudentID feature with $n=14$ students: $\text{IG} \approx 0.940$ bits, $\text{SplitInfo} = \log_2(14) = 3.807$ bits, $\text{GainRatio} = 0.247$ — dramatically penalising it as expected.

2.2.3 CART, ID3, and C4.5 — Detailed Comparison

Aspect	ID3	C4.5	CART
Developer	Quinlan (1986)	Quinlan (1993)	Breiman et al. (1984)

Criterion	Information Gain	Gain Ratio	Gini Impurity
Splits	Multi-way (one branch per value)	Multi-way with binary for continuous	Binary only (always two children)
Feature types	Categorical only	Categorical + continuous	Both (continuous via threshold search)
Pruning	None — grows fully	Post-pruning (pessimistic error)	Cost-complexity (α cross-validated)
Missing values	Cannot handle	Fractional sample allocation	Surrogate split selection
Output	Classification only	Classification only	Classification AND regression
scikit-learn	Not directly	Not directly	Yes — DecisionTree Classifier/Regressor

 **Note:** scikit-learn implements CART. The criterion parameter accepts 'gini' (default) or 'entropy' (which mimics Information Gain, not Gain Ratio). The splitter='best' parameter exhaustively searches all thresholds for continuous features.

2.2.4 Pruning Strategies

Pre-pruning (Early Stopping)

- `max_depth`: hard cap on tree depth. Prevents building overly complex trees.
- `min_samples_split`: minimum samples required to split an internal node (default: 2). Setting higher reduces tree complexity.
- `min_samples_leaf`: minimum samples required to be at a leaf node. Ensures leaves are 'trustworthy'.
- `min_impurity_decrease`: only split if the impurity reduction exceeds this threshold.

Post-pruning (Cost-Complexity Pruning — CART)

A fully grown tree T_0 is pruned by introducing a complexity penalty:

$$R_\alpha(T) = R(T) + \alpha \cdot |T|$$

Where $R(T)$ is the misclassification rate on training data, $|T|$ is the number of leaves, and $\alpha \geq 0$ is the complexity parameter. As α increases, the optimal tree becomes smaller. The optimal α is found by cross-validation, and scikit-learn exposes this through the `cost_complexity_pruning_path()` method.

Example — Pruning in Practice: Train a full tree (`max_depth=None`), call `clf.cost_complexity_pruning_path(X_train, y_train)` to get all $(\alpha, \text{impurity})$ pairs, then select α via `GridSearchCV`. This reliably finds a tree that generalises well without manual depth specification.

Section 2.3: Support Vector Machines — Mathematical Derivation

In Lecture 1 we stated the SVM optimisation problem. Here we derive it from first principles, convert the primal to the more useful dual form using Lagrange multipliers, establish the role of KKT conditions in defining support vectors, and provide practical guidance on hyperparameter selection.

2.3.1 The Primal Problem

Given a linearly separable training set $\{(x_1, y_1), \dots, (x_m, y_m)\}$ where $y_i \in \{-1, +1\}$, we seek the hyperplane $w \cdot x + b = 0$ that maximises the margin. The margin is $2/\|w\|$, so maximising the margin is equivalent to minimising $(1/2)\|w\|^2$:

$$\begin{aligned} \text{Minimise:} \quad & (1/2) \|w\|^2 \\ \text{Subject to:} \quad & y_i (w \cdot x_i + b) \geq 1 \quad \text{for all } i = 1, \dots, m \end{aligned}$$

This is a convex quadratic programming problem with m linear constraints (one per training example). The constraints ensure all training points are on the correct side of the margin boundaries $w \cdot x + b = \pm 1$.

2.3.2 The Lagrangian and the Dual Problem

We introduce Lagrange multipliers $\alpha_i \geq 0$, one per training constraint, to form the Lagrangian:

$$L(w, b, \alpha) = (1/2) \|w\|^2 - \sum_i \alpha_i [y_i (w \cdot x_i + b) - 1]$$

Setting the partial derivatives to zero at the saddle point:

$$\begin{aligned} \partial L / \partial w = 0 & \rightarrow w = \sum_i \alpha_i y_i x_i \\ \partial L / \partial b = 0 & \rightarrow \sum_i \alpha_i y_i = 0 \end{aligned}$$

Substituting these back into L , we obtain the dual problem — a function of α_i only:

$$\begin{aligned} \text{Maximise:} \quad & W(\alpha) = \sum_i \alpha_i - (1/2) \sum_i \sum_j \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) \\ \text{Subject to:} \quad & \sum_i \alpha_i y_i = 0 \quad \text{and} \quad \alpha_i \geq 0 \quad \text{for all } i \end{aligned}$$

The crucial observation is that the dual depends on the training data only through inner products $x_i \cdot x_j$. This is what enables the kernel trick: we can replace the inner product with a kernel function $K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$ without ever computing the high-dimensional feature map ϕ explicitly.

2.3.3 KKT Conditions and Support Vectors

The Karush-Kuhn-Tucker (KKT) complementary slackness conditions state that at the optimal solution:

$$\alpha_i [y_i(w \cdot x_i + b) - 1] = 0 \quad \text{for all } i$$

This has two implications:

- If $\alpha_i = 0$: the constraint is inactive — this training point lies strictly outside the margin and does not contribute to w . These are the 'easy' points.
- If $\alpha_i > 0$: the constraint is active — $y_i(w \cdot x_i + b) = 1$, meaning this point lies exactly on the margin boundary. These are the support vectors.

The decision boundary is therefore:

$$f(x) = \text{sign}(w \cdot x + b) = \text{sign}(\sum_i \alpha_i y_i (x_i \cdot x) + b)$$

The weight vector $w = \sum_i \alpha_i y_i x_i$ is a linear combination of only the support vectors (points where $\alpha_i > 0$). All other training examples have $\alpha_i = 0$ and are completely irrelevant to the decision boundary. This sparsity is a key practical advantage — prediction requires only a dot product with a small number of support vectors.

Example — Support Vector Sparsity: On a linearly separable dataset of 10,000 training examples, a typical SVM may have only 50–200 support vectors. Prediction for any new point requires computing 50–200 dot products, not 10,000 — a 50–200× speedup compared to methods that use all training data.

2.3.4 Soft Margin SVM

When data is not linearly separable, we introduce slack variables $\xi_i \geq 0$ that allow constraint violations:

$$\begin{aligned} \text{Minimise:} \quad & (1/2) ||w||^2 + C \cdot \sum_i \xi_i \\ \text{Subject to:} \quad & y_i(w \cdot x_i + b) \geq 1 - \xi_i \quad \text{and} \quad \xi_i \geq 0 \quad \text{for all } i \end{aligned}$$

The hyperparameter C controls the trade-off between maximising the margin and minimising training error:

- Large C : strong penalty for misclassification → narrow margin, few errors on training data → risk of overfitting on noisy data

- Small C: weak penalty → wide margin, allows more misclassification → more regularised model → may underfit

The dual of the soft-margin SVM is identical to the hard-margin dual except the constraint $0 \leq \alpha_i \leq C$ replaces $\alpha_i \geq 0$. The parameter C provides an upper bound on the Lagrange multipliers.

2.3.5 Hyperparameter Tuning: C and γ

Hyperparameter	Effect of Increasing	Practical Range	Default
C (regularisation)	Smaller margin, fewer train errors, more overfit risk	0.01 to 1000	1.0
γ (RBF kernel)	Each point's influence shrinks, boundary gets wiggly, overfit risk	0.0001 to 10	$1/n_{\text{features}}$

 **Note:** The parameters C and γ interact: high C + high γ → severe overfitting. High C + low γ → broad Gaussian, smooth boundary. Always search both jointly using GridSearchCV or RandomizedSearchCV.

2.3.6 Multi-Class SVMs

Standard SVMs are inherently binary classifiers. Two strategies extend them to K-class problems:

One-vs-Rest (OvR / OvA): Train K binary classifiers, one for each class against all others. For class k, positive examples are from class k, negatives are all other classes. Predict by choosing the class with the highest decision function score. Simpler but can suffer from class imbalance within each binary sub-problem.

One-vs-One (OvO): Train $K(K-1)/2$ binary classifiers for every pair of classes. Predict by majority vote across all pairwise classifiers. scikit-learn SVC uses OvO by default. More classifiers but each is balanced and trained on fewer samples (faster training per classifier).

2.3.7 Python Implementation Best Practices

```
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.model_selection import GridSearchCV, StratifiedKFold
```

```
# CRITICAL: SVM requires feature scaling
# Always use a Pipeline to prevent data leakage
pipe = Pipeline([
    ('scaler', StandardScaler()),    # z-score normalisation
    ('svm', SVC(kernel='rbf', probability=True))
])
```

```
# Joint grid search over C and gamma
param_grid = {
    'svm__C': [0.01, 0.1, 1, 10, 100, 1000],
    'svm__gamma': ['scale', 'auto', 0.001, 0.01, 0.1]
}
```

```
gs = GridSearchCV(pipe, param_grid,
                  cv=StratifiedKFold(5),
                  scoring='f1_weighted',
                  n_jobs=-1,
                  verbose=1)
gs.fit(X_train, y_train)
print('Best params:', gs.best_params_)
print('Best CV F1: ', gs.best_score_)
```

⚠ Warning: SVMs are NOT scale-invariant. If you forget StandardScaler, features measured in thousands (income) will dominate features measured in units (age), and the model will be poorly calibrated. Always scale inside a Pipeline.

Section 2.4: Training Neural Networks — Deep Dive

Training deep neural networks involves far more than running gradient descent. This section covers the fundamental failure modes (vanishing and exploding gradients), the optimisation algorithms that address them, and the regularisation techniques that prevent overfitting.

2.4.1 The Vanishing and Exploding Gradient Problems

Backpropagation computes the gradient of the loss L with respect to the weights in layer l by applying the chain rule repeatedly through all layers from L back to l :

$$\partial L / \partial w^l = \partial L / \partial a^L \cdot \prod_{k=l+1}^L (f'(z^k) \cdot w^k)$$

This product contains $L - l$ terms. Two pathological scenarios arise:

Vanishing Gradients

If $f'(z) < 1$ for all z (as is true for Sigmoid: $f'(z) = \sigma(z)(1-\sigma(z)) \leq 0.25$, and Tanh: $f'(z) = 1 - \tanh^2(z) \leq 1$), then the product of $L - l$ such terms decays exponentially to zero. Layers close to the input receive vanishingly small gradients and learn extremely slowly or not at all.

Solutions:

- Use ReLU activation: $f'(z) = 1$ for $z > 0$, eliminating gradient decay along positive activation paths
- Residual connections (skip connections in ResNets): provide an additive gradient highway — the gradient can flow unchanged through skip connections even if the residual path has vanishing gradients
- Xavier/He weight initialisation: choose initial weight scales to keep activation variances roughly constant across layers, preventing vanishing from the start
- LSTM gated units for sequential models: the cell state allows gradients to flow with multiplicative gating, avoiding repeated squashing

Exploding Gradients

If $\|W^k\| \cdot f'(z^k) > 1$ consistently across layers, the gradient grows exponentially, causing weight updates to become enormous. The training loss diverges and may produce NaN values.

Solutions:

- Gradient clipping (by norm): compute the global gradient norm $\|\nabla\|$. If it exceeds a threshold τ , scale the entire gradient: $\nabla \leftarrow \nabla \cdot \tau / \|\nabla\|$. This preserves gradient direction while controlling magnitude.
- Gradient clipping (by value): clamp each gradient element to $[-c, c]$. Simple but distorts gradient direction.
- Smaller learning rate: reduces the magnitude of weight updates
- Batch Normalisation: normalises pre-activations, which implicitly bounds the gradient magnitudes

2.4.2 Optimisation Algorithms

Stochastic Gradient Descent (SGD)

$$w \leftarrow w - \eta \cdot \nabla L(w; x_i, y_i)$$

The simplest form: compute the gradient on a single randomly sampled training example (or mini-batch of size B), update weights. Mini-batch SGD ($B = 32$ – 256) is the standard in practice — it approximates the full-batch gradient while being much faster.

- Advantages: simple, well-understood, good generalisation when combined with proper lr scheduling
- Disadvantages: sensitive to learning rate choice; slow convergence in loss landscape ravines

Momentum

$$\begin{aligned} v &\leftarrow \beta \cdot v - \eta \cdot \nabla L & (\beta \text{ typically } 0.9) \\ w &\leftarrow w + v \end{aligned}$$

Momentum accumulates a velocity vector in the direction of consistent gradient updates, damping oscillations perpendicular to the gradient direction (common in ravines) while accelerating along the loss valley. The effect is analogous to a ball rolling down a hill — it builds momentum in the consistent direction.

Adam (Adaptive Moment Estimation)

$$m_t = \beta_1 \cdot m_{t-1} + (1-\beta_1) \cdot \nabla L_t \quad (\text{first moment: velocity})$$

$$\begin{aligned}
 \mathbf{v}_t &= \beta_2 \cdot \mathbf{v}_{t-1} + (1-\beta_2) \cdot \nabla L_t^2 && \text{(second moment: squared gradient)} \\
 \hat{\mathbf{m}}_t &= \mathbf{m}_t / (1-\beta_1^t) && \hat{\mathbf{v}}_t = \mathbf{v}_t / (1-\beta_2^t) \quad \text{(bias correction)} \\
 \mathbf{w}_t &= \mathbf{w}_{t-1} - \eta \cdot \hat{\mathbf{m}}_t / (\sqrt{\hat{\mathbf{v}}_t} + \epsilon) && \text{(parameter update)}
 \end{aligned}$$

Adam adaptively scales the learning rate for each parameter: parameters with consistently large gradients (frequently updated) receive smaller updates; parameters with small or sparse gradients receive larger updates. Default hyperparameters: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, $\eta = 10^{-3}$.

- Adam is the default choice for most deep learning tasks — converges fast, robust to hyperparameters
- However, Adam can sometimes generalise slightly worse than well-tuned SGD+Momentum on image classification tasks, where SGD with cosine annealing is often preferred

2.4.3 Weight Initialisation

Initialising all weights to zero causes all neurons to compute identical activations and receive identical gradients — the symmetry breaking problem. Random initialisation breaks this symmetry.

Xavier/Glorot Initialisation: Designed for Sigmoid/Tanh: $w \sim \text{Uniform}(-\sqrt{6/(\text{fan_in}+\text{fan_out})}, +\sqrt{6/(\text{fan_in}+\text{fan_out})})$. Ensures activation variance is approximately preserved across layers.

He Initialisation: Designed for ReLU: $w \sim N(0, \sqrt{2/\text{fan_in}})$. Accounts for the fact that ReLU zeroes out half of its inputs on average, requiring larger initial weights to maintain variance.

2.4.4 Batch Normalisation

Batch Normalisation (BN), introduced by Ioffe and Szegedy (2015), normalises the pre-activation inputs to each layer within a mini-batch, then applies learnable affine transformation:

7. Compute mini-batch mean: $\mu_B = (1/m) \cdot \sum_i x_i$
8. Compute mini-batch variance: $\sigma^2_B = (1/m) \cdot \sum_i (x_i - \mu_B)^2$
9. Normalise: $\hat{x}_i = (x_i - \mu_B) / \sqrt{(\sigma^2_B + \epsilon)}$

10. Apply learnable scale and shift: $y_i = \gamma \cdot \hat{x}_i + \beta$ (γ and β are learned parameters)

The parameters γ and β allow the network to undo the normalisation if that is beneficial — they give BN the expressiveness to represent the original activation distribution if needed.

- Benefits: enables higher learning rates, acts as regulariser (often reduces need for Dropout), reduces sensitivity to weight initialisation, accelerates training by 2–14×
- At inference: use running mean and variance accumulated during training (not mini-batch statistics)

 **Note:** Batch Normalisation is applied between the linear transformation and the activation function: Linear → BatchNorm → Activation. Layer Normalisation (normalises across features, not the batch) is preferred for Transformer models where batch size may be 1.

2.4.5 Dropout

Dropout (Srivastava et al., 2014) is a regularisation technique that randomly zeroes out a fraction p of neuron activations during each forward pass in training:

$$\tilde{a}_i = m_i \cdot a_i \quad \text{where} \quad m_i \sim \text{Bernoulli}(1-p)$$

At each training step, a different random subset of neurons is dropped, effectively training an exponential ensemble of 2^n different 'thinned' networks (where n is the total number of neurons). This prevents neurons from co-adapting to each other's outputs — each neuron must learn features that are independently useful.

- At inference: all neurons are active, but outputs are scaled by $(1-p)$ to match the expected output during training ('inverted dropout' scaling is applied during training instead in modern frameworks)
- Typical dropout rates: $p = 0.5$ for fully-connected layers; $p = 0.1$ – 0.2 for convolutional layers (which are already regularised by weight sharing)
- SpatialDropout: drops entire feature maps in CNNs, respecting spatial correlation

Section 2.5: RNNs, LSTMs, and Transformer Models

2.5.1 Vanilla RNNs and Their Limitations

A Recurrent Neural Network processes a sequence $x_1, x_2, \dots, x_t$ by maintaining a hidden state h_t that summarises all information from the sequence up to time t :

$$\begin{aligned} h_t &= \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b) \\ \hat{y}_t &= \text{softmax}(W_y \cdot h_t + b_y) \end{aligned}$$

The hidden state acts as a fixed-size memory. The problem is that when we backpropagate through time (BPTT), we apply the chain rule T times through the recurrence — multiplying by W_h and $\tanh'$ at each step. When $\|W_h \cdot \tanh'\| < 1$, gradients vanish; when > 1 , they explode. In practice, vanilla RNNs fail to learn dependencies spanning more than 10–15 time steps.

2.5.2 LSTM — Full Gate Equations

The Long Short-Term Memory (LSTM), introduced by Hochreiter and Schmidhuber (1997), addresses the vanishing gradient problem by introducing a cell state C_t — a 'conveyor belt' that can carry information across many time steps with minimal modification.

The LSTM has four equations executed at each time step t , given input x_t and the previous hidden state h_{t-1} and cell state C_{t-1} :

Forget Gate

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

The forget gate decides what fraction of the previous cell state C_{t-1} to retain. Output $\in (0,1)$ per cell dimension. When $f_t \approx 0$ for a particular cell dimension: that information is discarded. When $f_t \approx 1$: the information is fully preserved.

Input Gate

$$\begin{aligned} i_t &= \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \\ \tilde{C}_t &= \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \end{aligned}$$

The input gate has two components: i_t (a sigmoid gate deciding which values to update) and $\tilde{C}_t$ (a tanh layer creating candidate values to add). The actual update uses both: $i_t \odot \tilde{C}_t$.

Cell State Update

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t$$

This is the key equation. The cell state $\mathbf{C}_t$ is updated by element-wise operations only: multiplication by the forget gate (selective retention) and addition of the gated candidate (selective writing). There is no non-linearity squashing the cell state itself — this is why gradients can flow across long sequences without vanishing: $\partial \mathbf{C}_t / \partial \mathbf{C}_{t-1} = \mathbf{f}_t$, which can be maintained near 1 by the forget gate learning to stay open.

Output Gate

$$\begin{aligned} \mathbf{o}_t &= \sigma(\mathbf{W}_o \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \\ \mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{C}_t) \end{aligned}$$

The output gate determines what part of the cell state to expose as the hidden state $\mathbf{h}_t$. The $\tanh$ squashes the cell state to $(-1, 1)$ before the output gate filters it.

Parameter Count for LSTM

An LSTM cell with input dimension d and hidden dimension h has 4 weight matrices ($\mathbf{W}_f, \mathbf{W}_i, \mathbf{W}_c, \mathbf{W}_o$) each of size $(h \times (d+h))$, plus 4 bias vectors of size h :

$$\text{Total parameters} = 4 \cdot h \cdot (d + h) + 4h = 4h(d + h + 1)$$

Example — LSTM Parameter Count: A language model LSTM with $d=256$ (embedding dimension) and $h=512$ (hidden size) has $4 \times 512 \times (256 + 512 + 1) = 4 \times 512 \times 769 = 1,576,960$ parameters per LSTM layer.

2.5.3 Gated Recurrent Unit (GRU)

The GRU (Cho et al., 2014) simplifies the LSTM by merging the cell state and hidden state, and combining the forget and input gates into a single update gate:

$$\begin{aligned} \mathbf{z}_t &= \sigma(\mathbf{W}_z \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t]) && \text{(update gate)} \\ \mathbf{r}_t &= \sigma(\mathbf{W}_r \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t]) && \text{(reset gate)} \\ \tilde{\mathbf{h}}_t &= \tanh(\mathbf{W} \cdot [\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t]) && \text{(candidate hidden state)} \\ \mathbf{h}_t &= (\mathbf{1} - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t && \text{(final hidden state)} \end{aligned}$$

GRUs have fewer parameters than LSTMs and train faster. On many tasks (machine translation, text classification, time series) they achieve comparable performance. The choice is empirical — try both.

2.5.4 The Attention Mechanism

The fundamental limitation of RNNs/LSTMs is the fixed-size hidden state bottleneck — all information from a long input sequence must be compressed into a single vector before decoding. Attention (Bahdanau et al., 2015) allows the decoder to selectively attend to different parts of the encoder output at each decoding step.

Scaled Dot-Product Attention

The Transformer architecture (Vaswani et al., 2017) uses the following formulation for all attention:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}(\mathbf{Q} \mathbf{K}^T / \sqrt{d_k}) \cdot \mathbf{V}$$

Where:

- $\mathbf{Q}$ (Query matrix): representations of what the current position is looking for (shape: $\text{seq_len} \times d_k$)
- $\mathbf{K}$ (Key matrix): representations of what each position offers as 'searchable' content (shape: $\text{seq_len} \times d_k$)
- $\mathbf{V}$ (Value matrix): the actual content to be retrieved if a position is attended to (shape: $\text{seq_len} \times d_v$)
- $\sqrt{d_k}$ scaling: dot products grow large in high dimensions, causing softmax to saturate (near-zero gradients); dividing by $\sqrt{d_k}$ restores the variance to approximately 1

The attention weights $\text{softmax}(\mathbf{Q} \mathbf{K}^T / \sqrt{d_k})$ form a $\text{seq_len} \times \text{seq_len}$ matrix — each position attends to every other position with a learned weight. Multiplying by $\mathbf{V}$ retrieves a weighted sum of value vectors.

Multi-Head Attention

$$\begin{aligned} \text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot \mathbf{W}^O \\ \text{head}_i &= \text{Attention}(\mathbf{Q} \cdot \mathbf{W}_i^Q, \mathbf{K} \cdot \mathbf{W}_i^K, \mathbf{V} \cdot \mathbf{W}_i^V) \end{aligned}$$

Running h attention heads in parallel allows the model to simultaneously attend to information from different representation subspaces at different positions. The outputs are concatenated and projected back to the model dimension. The Transformer's encoder uses 8–16 heads.

2.5.5 BERT — Bidirectional Encoder

BERT (Devlin et al., 2018) is a Transformer encoder model pre-trained on a massive text corpus using two self-supervised objectives:

Masked Language Modelling (MLM): 15% of input tokens are randomly masked. The model must predict the original token from context. Since the model sees both left and right context (unlike GPT which only sees left context), BERT learns bidirectional representations.

Next Sentence Prediction (NSP): Given two sentences, predict whether sentence B follows sentence A in the original document. Trains discourse-level understanding (though later research found NSP adds minimal value).

BERT-Base: 12 Transformer encoder layers, 768 hidden dimensions, 12 attention heads, 110M parameters. BERT-Large: 24 layers, 1024 dimensions, 16 heads, 340M parameters.

Fine-tuning adds a task-specific head on top of BERT's output and fine-tunes the entire model (including pre-trained weights) on a labelled downstream dataset — even with only a few hundred or thousand labelled examples, fine-tuned BERT typically outperforms purpose-built models trained on much more data.

Example — Text Classification with BERT: To classify customer reviews as positive/negative: take the [CLS] token embedding from BERT's final layer (a 768-dimensional vector summarising the entire input), add a linear classification head ($768 \rightarrow 2$), and fine-tune with a small learning rate ($2e-5$) for 3–5 epochs on the labelled reviews. This typically achieves 92–95% accuracy even with only 1,000 labelled examples.

Section 2.6: Advanced Clustering and Evaluation Methods

2.6.1 Gaussian Mixture Models (GMMs) and the EM Algorithm

k-Means assigns each point to exactly one cluster (hard assignment). Gaussian Mixture Models (GMMs) generalise this by assigning each point a probability of belonging to each cluster (soft assignment). A GMM models the data as a mixture of K Gaussian distributions:

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \cdot N(\mathbf{x} \mid \mu_k, \Sigma_k)$$

Where π_k is the mixing coefficient (prior probability of cluster k , satisfying $\sum \pi_k = 1$), μ_k is the mean, and Σ_k is the covariance matrix of the k -th Gaussian component.

The EM Algorithm for GMMs

Since we do not observe which component generated each point, we use Expectation-Maximisation (EM) — an iterative algorithm that alternates between inferring soft cluster assignments and updating parameters:

E-step (Expectation):

Compute the responsibility of component k for data point x_i — how much of x_i 's 'credit' belongs to component k :

$$r_{ik} = \pi_k \cdot N(\mathbf{x}_i \mid \mu_k, \Sigma_k) / \sum_j [\pi_j \cdot N(\mathbf{x}_i \mid \mu_j, \Sigma_j)]$$

$r_{ik} \in [0,1]$ and $\sum_k r_{ik} = 1$ for all i . Every point is softly assigned to all K components.

M-step (Maximisation):

Update parameters using the responsibilities as weights:

$$\begin{aligned} N_k &= \sum_i r_{ik} && \text{(effective number of points in component } k) \\ \mu_k &= (1/N_k) \sum_i r_{ik} \mathbf{x}_i && \text{(weighted mean)} \\ \Sigma_k &= (1/N_k) \sum_i r_{ik} (\mathbf{x}_i - \mu_k)(\mathbf{x}_i - \mu_k)^T && \text{(weighted covariance)} \\ \pi_k &= N_k / N && \text{(updated mixing coefficient)} \end{aligned}$$

Convergence:

Repeat $E \rightarrow M$ until the log-likelihood $\sum_i \log p(\mathbf{x}_i)$ stops increasing. EM is guaranteed to never decrease the likelihood at each step (it is a lower bound optimisation). However, it converges to a local optimum — multiple random restarts are recommended, keeping the solution with the highest final log-likelihood.

Model selection (choosing K): use the Bayesian Information Criterion (BIC) or Akaike Information Criterion (AIC) — both penalise model complexity, favouring simpler models that still fit well:

$$\text{BIC} = -2 \cdot \log(L^*) + d \cdot \log(N) \quad \text{where } d = \text{total parameters, } L^* = \text{maximum likelihood}$$

$$\text{AIC} = -2 \cdot \log(L^*) + 2d$$

Example — GMM vs k-Means: k-Means is a special case of GMM where all covariance matrices are constrained to be equal and spherical: $\Sigma_k = \sigma^2 I$. The soft assignments r_{ik} in the limit of $\sigma^2 \rightarrow 0$ become hard assignments (each point belongs entirely to the nearest centroid). GMMs can model elliptical clusters of different shapes and orientations, making them far more flexible than k-Means.

2.6.2 Spectral Clustering

Spectral clustering transforms the clustering problem into a graph partitioning problem, enabling it to find non-convex clusters (rings, crescents, interleaved spirals) that completely defeat k-Means.

Algorithm — Four Steps

11. Build the Similarity Graph: compute a pairwise similarity matrix W where $W_{ij} = \exp(-\|x_i - x_j\|^2 / (2\sigma^2))$ (Gaussian/RBF kernel). Alternatively, build a k-nearest-neighbour graph. σ controls the scale of similarity.
12. Compute the Graph Laplacian: D is the diagonal degree matrix ($D_{ii} = \sum_j W_{ij}$). The unnormalised Laplacian $L = D - W$. The normalised Laplacian $L_n = D^{-1/2} L D^{-1/2} = I - D^{-1/2} W D^{-1/2}$.
13. Eigendecomposition: compute the k eigenvectors corresponding to the k smallest eigenvalues of L (or L_n). The first eigenvector is always the constant vector (eigenvalue 0). The k smallest eigenvectors capture the cluster structure. Stack them column-wise to form the $n \times k$ embedding matrix U .
14. Cluster the Embeddings: apply k-Means to the rows of U . Each row is the 'spectral embedding' of one data point — in this space, the cluster structure is linearised and k-Means succeeds.

Why does it work? The eigenvalues of L reveal the number of connected components: if there are exactly k disconnected components in the graph, the k smallest eigenvalues are all 0 and

the corresponding eigenvectors are indicator functions of the components. For near-disconnected data (clusters with small inter-cluster connections), these eigenvectors still approximately separate the clusters in the low-eigenvalue subspace.

⚠ Warning: Spectral clustering requires computing eigenvalues of an $n \times n$ matrix — $O(n^3)$ time complexity. This makes it impractical for $n > 10,000$. For large datasets, use approximate methods (Nyström approximation, random projections) or switch to DBSCAN/HDBSCAN.

2.6.3 ROC vs Precision-Recall — When to Use Which

Both the ROC curve and the Precision-Recall (PR) curve evaluate classifier performance across all decision thresholds, but they respond differently to class imbalance.

ROC Curve Analysis

The ROC curve plots TPR (Recall) on the Y-axis against $FPR = FP/(FP+TN)$ on the X-axis. The FPR denominator includes True Negatives — when the negative class is large (imbalanced dataset), FPR is dominated by the vast TN count, making it appear artificially low even when the classifier produces many false positives.

Example — AUC-ROC is Misleading for Imbalance: Consider 10,000 samples: 100 positive, 9,900 negative. A classifier that flags all 100 positives but also flags 990 negatives achieves: $TPR = 100/100 = 1.0$, $FPR = 990/9900 = 0.10$. $AUC-ROC \approx 0.95$ — looks excellent! But $Precision = 100/(100+990) = 9.2\%$ — 91% of positive predictions are wrong.

Precision-Recall Curve

The PR curve plots Precision on the Y-axis against Recall on the X-axis. The baseline (a random classifier) is a horizontal line at $y = P/(P+N)$ — the class prevalence. For imbalanced datasets where $P/(P+N)$ is very small, the random baseline is near zero, making a good classifier's curve dramatically higher. PR curves are far more informative than ROC curves for imbalanced problems.

The Area Under the PR Curve (AUC-PR), also called Average Precision (AP), is the standard metric for information retrieval and object detection tasks.

2.6.4 Statistical Significance Testing

When comparing two classifiers, it is insufficient to report 'Model A achieves 89.3% accuracy and Model B achieves 88.7%.' Without a statistical test, we cannot know whether this difference is meaningful or just the result of random variation in the data.

McNemar's Test

McNemar's test is the standard test for comparing two classifiers evaluated on the same fixed test set. It focuses only on the cases where the two classifiers disagree:

$$\chi^2 = (|b - c| - 1)^2 / (b + c)$$

Where b = number of test examples where Model A is correct but Model B is wrong, and c = number where Model B is correct but Model A is wrong. The -1 is a continuity correction. Under H_0 (both models have the same error rate), χ^2 follows a chi-squared distribution with 1 degree of freedom. Reject H_0 if $p < 0.05$ ($\chi^2 > 3.84$).

 **Note:** McNemar's test only uses the disagreement cases (b and c) — the cases where both models agree (both right or both wrong) are irrelevant to testing whether one model is better. This makes it very sensitive to the disagreement structure.

k-Fold Cross-Validation t-Test

When comparing using k -fold CV scores, compute the per-fold difference $d_i = \text{score}_i(A) - \text{score}_i(B)$ and apply a paired t-test:

$$t = \bar{d} / (s_d / \sqrt{k}) \quad \text{where } \bar{d} = \text{mean}(d), s_d = \text{std}(d), k = \text{number of folds}$$

Compare t against the t-distribution with $k-1$ degrees of freedom. However, this test has inflated Type I error because CV folds overlap (share training data). Dietterich (1998) recommends the 5×2 cross-validation test instead, which uses 5 repetitions of 2-fold CV to estimate both variance components.

Friedman Test for Multiple Algorithms

When comparing $K \geq 3$ algorithms across multiple datasets, use the Friedman test (non-parametric ANOVA equivalent): rank algorithms within each dataset, then test whether the mean ranks differ significantly. If the Friedman test rejects H_0 , use post-hoc Nemenyi tests with Bonferroni correction for pairwise comparisons.

Example — Best Practice: A published comparison of 5 algorithms on 12 datasets: (1) Rank algorithms within each dataset, (2) Compute mean ranks across datasets, (3) Apply

Friedman test (H_0 : all algorithms perform equally), (4) If rejected, apply Nemenyi post-hoc test to identify which pairs differ significantly. Include a critical difference (CD) diagram in the paper.

Section 2.7: Feature Importance and Model Interpretation

Understanding which features drive a model's predictions is increasingly important — both for debugging models and for meeting legal requirements (GDPR's right to explanation).

2.7.1 Permutation Importance

Permutation importance (Breiman, 2001) is a model-agnostic technique: for each feature j , randomly shuffle the values of feature j in the validation set (breaking the relationship between j and the target), evaluate model performance, and record the drop in performance. Features whose shuffling causes large performance drops are important.

$$\text{Importance}(j) = \text{Performance}(\text{original}) - \text{Performance}(\text{shuffled feature } j)$$

- Works for any model: neural networks, SVMs, random forests, k-NN
- Accounts for feature interactions (unlike filter-based methods)
- Can be misleading when features are highly correlated — shuffling one correlated feature may have minimal effect because the model still has access to the correlation partner

2.7.2 SHAP Values

SHAP (SHapley Additive exPlanations, Lundberg & Lee 2017) is the current gold standard for model explanation. It assigns each feature a Shapley value — borrowed from cooperative game theory — representing that feature's average marginal contribution to the prediction across all possible subsets of features.

$$\phi_j(f, x) = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!(|F|-|S|-1)!}{|F|!} \cdot [f(S \cup \{j\}) - f(S)]$$

Computing this exactly requires 2^n model evaluations (exponential in n features). TreeSHAP (for tree ensembles) computes exact Shapley values in polynomial time. KernelSHAP provides model-agnostic approximate Shapley values in $O(n^2)$ evaluations per prediction.

SHAP values are additive: $f(x) = E[f(X)] + \sum_j \phi_j(x)$. The prediction is decomposed into the base value (expected model output) plus each feature's contribution. Positive ϕ_j means feature j pushed the prediction above the average; negative means it pushed below.

```
import shap
```

```
# TreeSHAP for Random Forest (fast, exact)
explainer = shap.TreeExplainer(rf_model)
shap_values = explainer.shap_values(X_test) # shape: [n_samples,
n_features]

# Global feature importance (mean |SHAP value| across test set)
shap.summary_plot(shap_values, X_test, feature_names=feature_names)

# Local explanation for a single prediction
shap.waterfall_plot(explainer(X_test[0]))
```

2.7.3 LIME

LIME (Local Interpretable Model-Agnostic Explanations, Ribeiro et al., 2016) explains individual predictions of any black-box model by locally approximating the model's behaviour around the prediction point with an interpretable linear model:

15. Select the instance to explain: x^*
16. Sample perturbed instances around x^* (for tabular data: randomly zero out features; for text: remove words; for images: mask superpixels)
17. Obtain predictions for all perturbed instances from the black-box model
18. Weight perturbed instances by their proximity to x^* (using an exponential kernel)
19. Fit a weighted sparse linear regression on the perturbed instances
20. The linear model's coefficients are the LIME explanation — they show which features pushed the prediction in which direction locally

 **Note:** LIME explanations are local and unstable: running LIME twice on the same instance with different random seeds may give noticeably different explanations. SHAP values, by contrast, are uniquely defined by the game theory axioms. Use SHAP for reliability; LIME can still be useful for quick, human-readable explanations for text and images.

Section 2.8: Handling Imbalanced Datasets — Complete Guide

Class imbalance — where one class has far fewer samples than others — is ubiquitous in practice: fraud detection (0.1% positive), medical diagnosis (rare diseases), fault detection, cybersecurity intrusion detection. Standard classifiers optimise accuracy, which is dominated by the majority class in imbalanced settings.

2.8.1 Diagnosing the Problem

Before applying any correction, quantify the imbalance and its effect:

- Imbalance ratio: $\text{minority_count} / \text{majority_count}$. Ratios below 0.01 (1:100) require aggressive treatment.
- Evaluate with appropriate metrics: Recall (sensitivity), F1-score, Matthews Correlation Coefficient (MCC), AUC-PR. Never use accuracy alone.
- Establish a baseline: a classifier that predicts the majority class for everything — what accuracy, F1, and AUC-PR does it achieve?

2.8.2 Resampling Techniques

Random Under-sampling

Randomly remove majority class samples until the dataset is balanced. Simple and fast, but risks discarding informative samples from the majority class. Use when the majority class is very large and redundant.

```
from imblearn.under_sampling import RandomUnderSampler
rus = RandomUnderSampler(sampling_strategy=0.5, random_state=42)
# sampling_strategy=0.5 → minority:majority = 1:2
X_res, y_res = rus.fit_resample(X_train, y_train)
```

SMOTE — Synthetic Minority Over-sampling Technique

SMOTE (Chawla et al., 2002) generates synthetic minority class samples by interpolating between existing minority class samples:

21. For each minority class sample x , find its k nearest minority class neighbours (typically $k=5$)

22. Choose one neighbour x' randomly

23. Generate synthetic sample: $x_{\text{new}} = x + \lambda \cdot (x' - x)$ where $\lambda \sim \text{Uniform}(0,1)$

This creates new samples along the line segments connecting minority class neighbours — interpolating in feature space rather than simply duplicating existing samples.

```
from imblearn.over_sampling import SMOTE
smote = SMOTE(k_neighbors=5, random_state=42)
X_res, y_res = smote.fit_resample(X_train, y_train)
```

⚠ Warning: Never apply SMOTE before splitting into train/test. Always fit SMOTE on the training set only (inside a cross-validation fold), otherwise synthetic samples from the training set contaminate the test set.

2.8.3 Algorithmic Approaches

Class Weights

The simplest and often most effective approach: multiply the loss for minority class examples by a weight inversely proportional to class frequency. No data augmentation, no resampling — just a weighted loss function.

```
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier

# class_weight='balanced' automatically computes:
# w_c = n_samples / (n_classes * n_samples_in_class_c)
svm = SVC(class_weight='balanced')
rf = RandomForestClassifier(class_weight='balanced_subsample')
```

Threshold Tuning

By default, classifiers use a decision threshold of 0.5 (classify as positive if $P(y=1|x) > 0.5$). Moving the threshold lower increases recall (catches more positives) at the cost of lower precision (more false alarms). The optimal threshold depends on the business cost of false negatives vs false positives.

```
from sklearn.metrics import precision_recall_curve, f1_score
import numpy as np

# Get probability scores on validation set
```

```
scores = clf.predict_proba(X_val)[:, 1]
precisions, recalls, thresholds = precision_recall_curve(y_val, scores)

# Find threshold maximising F1
f1_scores = 2 * (precisions * recalls) / (precisions + recalls + 1e-8)
optimal_idx = np.argmax(f1_scores)
optimal_threshold = thresholds[optimal_idx]
y_pred = (scores >= optimal_threshold).astype(int)
```

Section 2.9: Advanced Association Rule Measures

Support, confidence, and lift (introduced in Lecture 1) are the most commonly used measures, but they have weaknesses. This section introduces more sophisticated interestingness measures and guidelines for filtering the typically enormous number of generated rules.

2.9.1 Limitations of Basic Measures

Problems with Confidence

High confidence can be misleading when the consequent has high support. If 80% of all transactions contain milk, a rule $X \rightarrow \text{milk}$ will have confidence $\geq 80\%$ regardless of X . Lift accounts for this: $\text{Lift}(X \rightarrow \text{milk}) = 80\%/80\% = 1.0$ if X and milk are truly independent.

Problems with Lift

Lift is symmetric: $\text{Lift}(X \rightarrow Y) = \text{Lift}(Y \rightarrow X)$ always. However, rules $X \rightarrow Y$ and $Y \rightarrow X$ have very different practical implications. A rule is actionable only in the direction where you can intervene — for market basket analysis, you can only recommend Y given X is already in the basket, not vice versa.

2.9.2 Additional Interestingness Measures

Measure	Formula	Range	Key Property
Conviction	$(1 - \text{Supp}(Y)) / (1 - \text{Conf}(X \rightarrow Y))$	$[0, \infty)$	Directional; $\rightarrow \infty$ for perfect rules; $= 1$ for independence
Leverage	$\text{Supp}(X \cup Y) - \text{Supp}(X) \cdot \text{Supp}(Y)$	$[-0.25, 0.25]$	> 0 : more co-occurrence than chance
Kulczynski (Kulc)	$[\text{Conf}(X \rightarrow Y) + \text{Conf}(Y \rightarrow X)] / 2$	$[0, 1]$	Symmetric; robust to support

			imbalance
Cosine	$\text{Supp}(X \cup Y) / \sqrt{(\text{Supp}(X) \cdot \text{Supp}(Y))}$	[0, 1]	Geometric mean of both confidence s
Imbalance Ratio (IR)	$ \text{Supp}(X) - \text{Supp}(Y) / (\text{Supp}(X) + \text{Supp}(Y) - \text{Supp}(X \cup Y))$	[0, 1]	= 0: balanced; = 1: perfect implication
Zhang's metric	$(\text{Supp}(X \cup Y) - \text{Supp}(X) \cdot \text{Supp}(Y)) / \max(\text{Supp}(X \cup Y)(1 - \text{Supp}(X)), \text{Supp}(X) \cdot (\text{Supp}(Y) - \text{Supp}(X \cup Y)))$	[-1, 1]	Balanced; handles support extremes

 **Note:** In practice, use Kulc and Imbalance Ratio together: Kulc measures the overall strength of the association; IR indicates whether the association is balanced (both items imply each other equally) or asymmetric (one implies the other much more strongly than vice versa). A rule with high Kulc and low IR is a balanced, strong association.

2.9.3 Rule Filtering Strategy

Mining with low minimum support typically generates thousands or millions of rules. A systematic filtering pipeline is essential before presenting results to stakeholders:

24. Remove trivially obvious rules (e.g., bread → butter is known without data mining)
25. Remove rules with $\text{Lift} \leq 1$ (no positive correlation — the items are independent or negatively correlated)
26. Remove rules where the consequent has very high support (> 0.8) — confidence is misleading
27. Apply minimum Conviction > 1.1 to ensure genuine directional implication
28. Sort remaining rules by Kulczynski score descending to prioritise the strongest associations
29. Apply domain knowledge: a data mining team should review the top-50 rules with business stakeholders to identify actionable insights

Section 2.10: End-to-End ML Pipeline — Best Practices

A complete, production-quality machine learning pipeline integrates preprocessing, feature engineering, model training, evaluation, and deployment into a coherent, reproducible, and leak-proof workflow.

2.10.1 The Data Leakage Problem

Data leakage occurs when information from the test set contaminates the training process — producing models that appear to perform well in evaluation but fail in production. The most common sources are:

- Fitting preprocessing on the full dataset (including test samples) before splitting — a `StandardScaler` fit on all data learns test set statistics
- Feature selection on the full dataset — selecting features correlated with the target in test samples
- SMOTE applied before splitting — synthetic samples generated from test set neighbours end up in the training set
- Target encoding computed on the full dataset — test set target values influence training features

The solution is to wrap all preprocessing inside a scikit-learn `Pipeline`. Pipelines fit all transformers only on training data and apply them to test data without refitting.

2.10.2 Complete Pipeline Implementation

```
import numpy as np, pandas as pd
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold, cross_validate
from sklearn.metrics import classification_report, roc_auc_score
```

```
# 1. Define feature column groups
num_cols = ['age', 'income', 'credit_score']
cat_cols = ['gender', 'occupation', 'region']
```



```
# 2. Build preprocessing pipelines for each group
num_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])
cat_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(handle_unknown='ignore', sparse=False))
])
preprocessor = ColumnTransformer([
    ('num', num_transformer, num_cols),
    ('cat', cat_transformer, cat_cols)
])
```

```
# 3. Full pipeline with classifier
pipe = Pipeline([
    ('prep', preprocessor),
    ('clf', RandomForestClassifier(n_estimators=200,
                                   class_weight='balanced',
                                   random_state=42))
])
```

```
# 4. Stratified cross-validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
results = cross_validate(pipe, X, y, cv=cv,
    scoring=['f1_weighted', 'roc_auc', 'precision_weighted',
    'recall_weighted'],
    return_train_score=True)
```

```
# 5. Report results
for metric in ['f1_weighted', 'roc_auc']:
    test_scores = results[f'test_{metric}']
    train_scores = results[f'train_{metric}']
    print(f'{metric}:')
    print(f'  Train: {train_scores.mean():.3f} ± {train_scores.std():.3f}')
    print(f'  Test: {test_scores.mean():.3f} ± {test_scores.std():.3f}')
```


Revision Guide — Lecture 2

10.1 Key Concepts Checklist

Section 2.1 — Naive Bayes

- Derive the Naive Bayes decision rule from Bayes' theorem using conditional independence assumption
- Explain and apply Laplace smoothing: formula $P(w_i|c) = (\text{count} + \alpha) / (\text{total} + \alpha|V|)$, reason for α
- Work the spam classification example end-to-end from priors through log-scores to decision
- Distinguish Gaussian, Multinomial, and Bernoulli Naive Bayes — when to use each

Section 2.2 — Decision Trees

- Compute entropy $H(S) = -\sum p_i \log_2 p_i$ for any class distribution
- Compute Information Gain and identify the best split (Tennis dataset: Outlook wins with 0.247 bits)
- Explain Gain Ratio and why it corrects for high-cardinality features (StudentID problem)
- Compare CART vs ID3 vs C4.5 across 7 dimensions (criterion, splits, pruning, missing values, output)
- Describe cost-complexity pruning: $R_\alpha(T) = R(T) + \alpha \cdot |T|$, how α is selected by CV

Section 2.3 — SVM

- State the primal SVM optimisation problem and its constraints
- Derive the Lagrangian and the dual problem; explain why the dual depends only on inner products
- State KKT conditions and derive the definition of support vectors from complementary slackness
- Explain the effect of C (small $C \rightarrow$ wide margin, underfitting; large $C \rightarrow$ narrow margin, overfitting)
- Explain the effect of γ in RBF kernel (small $\gamma \rightarrow$ wide Gaussian; large $\gamma \rightarrow$ narrow Gaussian)
- Distinguish OvR and OvO multi-class SVM strategies

Section 2.4 — Neural Network Training

- Explain vanishing gradient: chain rule product decays for Sigmoid; solutions: ReLU, skip connections, He init
- Explain exploding gradient and its solution: gradient clipping by norm
- Write update equations for SGD, Momentum, and Adam; state default hyperparameters for Adam
- Write the 4-step Batch Normalisation algorithm; explain learnable γ and β parameters
- Explain Dropout: zeroing with probability p during training, scaling by $(1-p)$ at inference

Section 2.5 — RNNs and Transformers

- Write all 4 LSTM gate equations: forget f_t , input i_t , candidate $\tilde{C}_t$, cell update C_t , output o_t , hidden h_t
- Explain why LSTM solves vanishing gradient: cell state uses only $\odot$ and $+$, no squashing
- Write scaled dot-product attention formula: $\text{softmax}(QK^T/\sqrt{d_k}) \cdot V$
- Explain Q, K, V roles and why $\sqrt{d_k}$ scaling is necessary
- Describe BERT: masked LM pre-training, bidirectionality, fine-tuning with task-specific head

Section 2.6 — Advanced Clustering and Evaluation

- Write GMM E-step: responsibility r_{ik} formula
- Write GMM M-step: N_k , μ_k , Σ_k , π_k update equations
- Describe spectral clustering in 4 steps: similarity graph $\rightarrow$ Laplacian $\rightarrow$ eigendecomposition $\rightarrow$ k-Means
- Explain why AUC-ROC is misleading for imbalanced data; when to use AUC-PR instead
- State McNemar's test statistic and when to use it vs paired t-test vs Friedman test

10.2 Sample Examination Questions with Model Answers

Question 1 (12 marks)

Using the training corpus from Section 2.1.4, classify the email 'win free money' as Spam or Ham. Show all steps including Laplace smoothing with $\alpha = 1$.

Model Answer:

Step 1 — Priors: $P(\text{Spam}) = 2/5 = 0.40$, $P(\text{Ham}) = 3/5 = 0.60$.

Step 2 — Vocabulary $|V| = 13$. Spam tokens = 7, Ham tokens = 10. Denominators: Spam = $7+13 = 20$, Ham = $10+13 = 23$.

Step 3 — Likelihoods with Laplace smoothing ($\alpha=1$):

- $P(\text{win}|\text{Spam}) = (1+1)/20 = 2/20 = 0.100$ $P(\text{win}|\text{Ham}) = (0+1)/23 = 1/23 = 0.043$
- $P(\text{free}|\text{Spam}) = (2+1)/20 = 3/20 = 0.150$ $P(\text{free}|\text{Ham}) = (1+1)/23 = 2/23 = 0.087$
- $P(\text{money}|\text{Spam}) = (1+1)/20 = 2/20 = 0.100$ $P(\text{money}|\text{Ham}) = (0+1)/23 = 1/23 = 0.043$

Step 4 — Log-scores:

- $\log \text{score}(\text{Spam}) = \log(0.40) + \log(0.100) + \log(0.150) + \log(0.100) = -0.916 + (-2.303) + (-1.897) + (-2.303) = -7.419$
- $\log \text{score}(\text{Ham}) = \log(0.60) + \log(0.043) + \log(0.087) + \log(0.043) = -0.511 + (-3.147) + (-2.441) + (-3.147) = -9.246$

Decision: $-7.419 > -9.246 \rightarrow \text{SPAM}$. Without smoothing, $P(\text{win}|\text{Ham})=0$ and $P(\text{money}|\text{Ham})=0$ would make $P(\text{Ham}|\text{email})=0$, preventing classification.

Question 2 (10 marks)

Compute Information Gain for splitting on Wind (Weak: 6 Yes, 2 No; Strong: 3 Yes, 3 No) from the root node $H(S) = 0.940$ bits. Is Outlook (IG = 0.247) still the better split?

Model Answer:

$$H(\text{Weak}) = -(6/8)\log_2(6/8) - (2/8)\log_2(2/8) = -0.75 \times (-0.415) - 0.25 \times (-2.000) = 0.311 + 0.500 = 0.811 \text{ bits}$$

$$H(\text{Strong}) = -(3/6)\log_2(3/6) - (3/6)\log_2(3/6) = -0.5 \times (-1.000) - 0.5 \times (-1.000) = 1.000 \text{ bits}$$

$$IG(\text{Wind}) = 0.940 - [(8/14) \times 0.811 + (6/14) \times 1.000] = 0.940 - [0.464 + 0.429] = 0.940 - 0.893 = 0.047 \text{ bits}$$

Comparison: $IG(\text{Outlook})=0.247 \gg IG(\text{Wind})=0.047$. Yes, Outlook is overwhelmingly the better first split. Wind provides very little discriminatory power (nearly half its branches are 50:50 splits). Outlook should be the root node.

Question 3 (12 marks)

Explain why vanishing gradients occur in deep networks that use Sigmoid activations. Describe THREE solutions, explaining the mechanism by which each addresses the problem.

Model Answer:

Cause: Backpropagation applies the chain rule through L layers: $\partial L / \partial w^l = \partial L / \partial a^l \cdot \prod (f'(z^k) \cdot W^k)$. For Sigmoid, $f'(z) = \sigma(z)(1-\sigma(z)) \in (0, 0.25]$. The product of $L-l$ such terms decays exponentially to zero. Early layers receive effectively zero gradient and cannot learn.

Solution 1 — ReLU Activation: $f'(z) = 1$ for $z > 0$, 0 for $z < 0$. The gradient does not decay through positive activations, maintaining gradient magnitude across layers.

Solution 2 — Residual Connections (ResNets): add a skip connection $x^{l+1} = F(x^l) + x^l$. The gradient through the skip path is $\partial L / \partial x^l = \partial L / \partial x^{l+1} \cdot (\partial F / \partial x^l + 1)$. The additive 1 ensures the gradient always has a component that reaches early layers unchanged, regardless of the residual path's gradient.

Solution 3 — He Weight Initialisation: initialise weights $w \sim N(0, \sqrt{(2/\text{fan_in})})$. This is calibrated so that activation variances are approximately 1 through each layer with ReLU, preventing activations from shrinking to zero (which also zeroes gradients) from the very start of training.

Question 4 (10 marks)

Derive the LSTM forget gate equation and explain what happens to the cell state when (a) $f_t \approx 0$ and (b) $f_t \approx 1$. Why does this design help with the vanishing gradient problem?

Model Answer:

Forget gate: $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$. Concatenate the previous hidden state h_{t-1} and current input x_t , apply a linear transformation with weight matrix W_f and bias b_f , pass through sigmoid to get element-wise values in $(0,1)$.

Cell state update: $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$.

(a) $f_t \approx 0$: $C_t \approx 0 \cdot C_{t-1} + i_t \odot \tilde{C}_t = i_t \odot \tilde{C}_t$. Old cell state is completely erased. The LSTM has 'forgotten' whatever was stored — appropriate when context changes completely (e.g., at a sentence boundary).

(b) $f_t \approx 1$: $C_t \approx C_{t-1} + i_t \odot \tilde{C}_t$. Old cell state passes through unchanged (plus new information added). Long-term dependencies are preserved.

Vanishing gradient: $\partial C_t / \partial C_{t-1} = f_t$. When the network learns to keep $f_t \approx 1$, this partial derivative is ≈ 1 — the gradient flows unchanged backward through many time steps without decay. This contrasts with vanilla RNNs where the gradient passes through $\tanh' (\leq 1)$ and W at every step.

Question 5 (8 marks)

A model achieves AUC-ROC = 0.99 on a medical screening dataset where 0.1% of patients have the target disease. Does this guarantee the model effectively detects the disease? Propose two better evaluation metrics with justification.

Model Answer:

No. With 0.1% prevalence, a model flagging 1,000 patients from 10,000 screened: if 10 are true positives (recall = 10%), $FPR = 990/9900 = 0.10$. AUC-ROC incorporates the TN count in FPR — with 9,900 negatives, even 990 false positives give a small FPR, making ROC look excellent while precision is only 1%. The model finds almost no disease cases.

Better metric 1 — AUC-PR (Average Precision): the PR curve plots precision vs recall, with baseline at $y = \text{prevalence} = 0.001$. A good classifier must achieve high precision at high recall — both denominators involve the small positive set, making inflated TN counts irrelevant.

Better metric 2 — Recall at fixed Precision (e.g., $R@P=0.5$): in medical screening, missing a true case (false negative) is typically far more costly than a false alarm. Setting minimum precision = 50% (acceptable false alarm rate) and maximising recall within that constraint directly addresses the clinical goal.

Question 6 (13 marks)

Compare GMM with k-Means: (a) What additional flexibility does GMM provide? (b) Write the E-step equation for GMM and explain what each term means. (c) When would you prefer k-Means?

Model Answer:

(a) Flexibility: GMM uses a full covariance matrix Σ_k per component — clusters can be elliptical (any shape, orientation, and scale). k-Means implicitly assumes $\Sigma_k = \sigma^2 I$ (spherical, equal-size clusters). GMM also provides soft assignments $r_{ik} \in [0,1]$ — each point's probability of belonging to each cluster — versus k-Means' hard assignments. GMM uses BIC/AIC for model selection (choosing K); k-Means uses the elbow/silhouette methods.

(b) E-step equation: $r_{ik} = \pi_k \cdot N(x_i | \mu_k, \Sigma_k) / \sum_j [\pi_j \cdot N(x_i | \mu_j, \Sigma_j)]$

- π_k : prior mixing coefficient — how 'important' component k is overall
- $N(x_i | \mu_k, \Sigma_k)$: Gaussian likelihood — how well x_i fits into component k's distribution
- Denominator: normalisation ensuring r_{ik} sums to 1 over all K components for each point x_i
- r_{ik} : the posterior probability that x_i was generated by component k

(c) Prefer k-Means when: the dataset is very large (k-Means is $O(n \cdot k \cdot l)$ vs GMM's $O(n \cdot K^2 \cdot d^2)$ for full covariance); clusters are approximately spherical and equal-sized; hard assignments are required (e.g., for downstream discrete processing); or when speed matters more than precision (k-Means is 10–100× faster).

10.3 Glossary of New Terms from Lecture 2

Term	Definition
Adam	Adaptive Moment Estimation — optimiser combining first (momentum) and second (variance) moment estimates
Attention	Mechanism allowing models to weight the relevance of different positions when processing a sequence
AUC-PR	Area Under the Precision-Recall Curve — preferred evaluation metric for imbalanced classification
Batch Normalisation	Normalises pre-activations within mini-batch; adds learnable γ and β ; accelerates training
BERT	Bidirectional Encoder Representations from Transformers — contextual language model pre-trained by MLM
BIC / AIC	Bayesian/Akaike Information Criterion — penalised likelihood for model selection (choosing K in GMM)
Conviction	Directional interestingness measure:

	$(1 - \text{Supp}(Y)) / (1 - \text{Conf}(X \rightarrow Y)); \rightarrow \infty$ for perfect rules
Cost-Complexity Pruning	Post-pruning method: minimise $R_\alpha(T) = R(T) + \alpha T $; α chosen by cross-validation
Data Leakage	Test set information contaminating training, causing overly optimistic evaluation
Dropout	Regularisation: randomly zero neuron activations during training with probability p
EM Algorithm	Expectation-Maximisation: alternates E-step (soft assignments) and M-step (parameter updates)
Exploding Gradient	Gradient grows exponentially during backprop; fixed by gradient clipping
Forget Gate	LSTM gate $f_t = \sigma(\dots) \in (0,1)$ controlling what fraction of cell state C_{t-1} to retain
Gain Ratio	Information Gain normalised by SplitInfo; corrects bias towards high-cardinality features
Gaussian Mixture Model	Probabilistic model: data is mixture of K Gaussians with parameters π_k, μ_k, Σ_k
GRU	Gated Recurrent Unit — simplified LSTM with update and reset gates
He Initialisation	$w \sim N(0, \sqrt{(2/\text{fan_in})})$ — correct initial weight scale for ReLU activations
KKT Conditions	Karush-Kuhn-Tucker optimality conditions; support vectors satisfy $\alpha_i[y_i(w \cdot x_i + b) - 1] = 0$
Kulczynski (Kulc)	Symmetric association measure: mean of both directional confidences
Lagrangian Dual	Alternative SVM formulation depending only on inner products $x_i \cdot x_j$, enabling kernel trick
Laplace Smoothing	Add α pseudo-counts to all word counts to prevent zero probabilities in Naive Bayes
LIME	Local Interpretable Model-Agnostic Explanations — local linear surrogate for any black-box prediction
McNemar's Test	$\chi^2 = (b - c - 1)^2 / (b + c)$; tests if two classifiers on same test set differ significantly
MLM	Masked Language Modelling — BERT pre-training task: predict 15% of randomly masked tokens
Momentum	Optimiser that accumulates gradient velocity in consistent direction; dampens oscillations
Multi-Head Attention	Running h attention heads in parallel; captures

	different representation subspaces
Permutation Importance	Feature importance by measuring performance drop when feature j is randomly shuffled
SHAP	SHapley Additive exPlanations — assigns each feature its average marginal Shapley contribution
SMOTE	Synthetic Minority Over-sampling: interpolates between minority class neighbours
Soft Margin SVM	SVM allowing misclassification via slack variables ξ_i ; controlled by hyperparameter C
Spectral Clustering	Graph-based clustering: build similarity graph $\rightarrow$ Laplacian eigendecomposition $\rightarrow$ k-Means on embeddings
Support Vectors	Training points on the margin boundary where $\alpha_i > 0$; define the SVM decision boundary
Vanishing Gradient	Gradient decays exponentially through deep layers with Sigmoid; fixed by ReLU/ResNets/LSTMs
Xavier Initialisation	Glorot: $w \sim \text{Uniform}(\pm\sqrt{6/(\text{fan_in}+\text{fan_out})})$; correct for Sigmoid/Tanh activations

— End of CSD622 Lecture 2 Notes —

Dr A.J Fofanah · CSD622 Pattern Recognition and Data Mining · Lecture 2